



Fullstack Web Development

Javascript - Part 01

Instructor: Ms. Liev Nova



090667393



What is Javascript?

JavaScript is the **world’s most popular programming language** used for building interactive websites.

- It is a high-level, interpreted scripting language
- Runs directly in the web browser
- Makes websites dynamic and interactive

Without JavaScript, websites are static (no interaction)

Why Study JavaScript?

To build a complete website, you need:

- **HTML** : Structure (content)
- **CSS** : Design (style & layout)
- **JavaScript** : Behavior (interaction & logic)

JavaScript is what makes your website **"alive"**

What Can JavaScript Do?

JavaScript makes websites **interactive and dynamic:**

- **Manipulate HTML & CSS**
  - Change text, images, styles dynamically
  - **Example:** Show/hide elements
- **Handle User Interaction**
  - Respond to clicks, typing, scrolling
  - **Example:** Button click → show message
- **Form Validation**
  - Check user input before submitting forms
  - **Example**: Email must be valid
- **Create Animations**
  - Sliders, transitions, interactive UI effects
- **Update Content Without Reload**
  - Load new data using APIs (AJAX / Fetch)

What Can JavaScript Do?

Simple Example

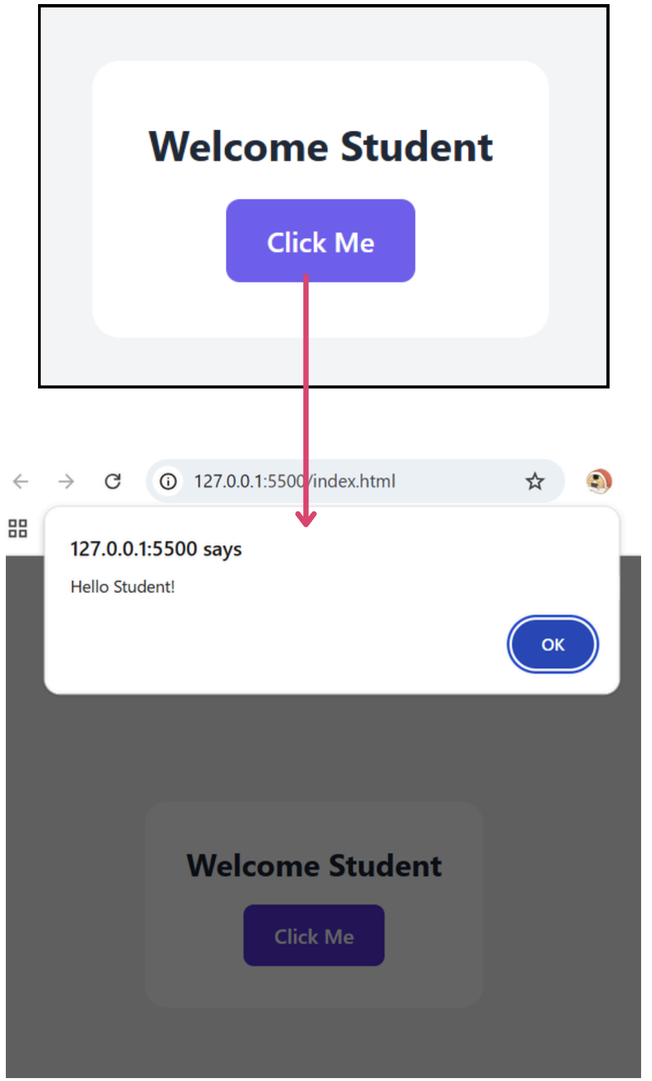
```
<body class="min-h-screen bg-gray-100 flex items-center justify-center">

    <div class="bg-white p-8 rounded-2xl text-center">
        <h1 class="text-2xl font-bold mb-4 text-gray-800">Welcome Student</h1>

        <button id="alertBtn"
            class="bg-indigo-500 hover:bg-indigo-600 text-white px-6 py-3 rounded-lg font-medium transition">
            Click Me
        </button>
    </div>

    <script>
        document.getElementById("alertBtn").addEventListener("click", function () {
            alert("Hello Student!");
        });
    </script>

</body>
```



JavaScript Syntax

JavaScript syntax defines how we write code.

Basic Example:

```
<body class="bg-white min-h-screen flex items-center justify-center">

    <div class="bg-white p-8 rounded-2xl border text-center">
        <h1 class="text-2xl font-bold text-gray-800 mb-4">
            JavaScript Syntax Example
        </h1>

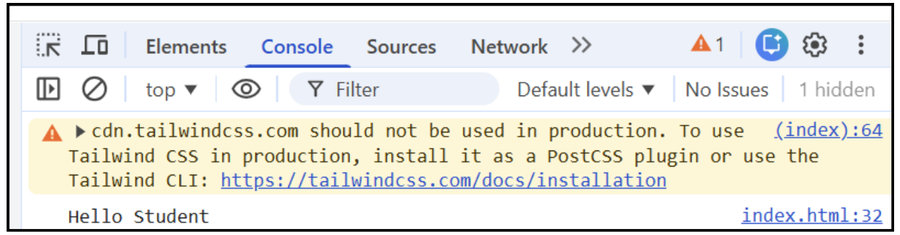
        <button onclick="showMessage()"
            class="bg-green-500 hover:bg-green-600 text-white px-6 py-3 rounded-lg font-semibold transition">
            Run Code
        </button>

        <p class="mt-4 text-sm text-gray-500">
            Open Console (F12) to see output
        </p>
    </div>

    <script>
        function showMessage() {
            let message = "Hello Student";
            console.log(message);
        }
    </script>

</body>
```





JavaScript Syntax

JavaScript syntax defines how we write code.

Syntax Rules:

- JavaScript is **case-sensitive**
- Statements usually end with **;**
- Use **{ }** for code blocks
- Use **//** for comments

```
<script>
    function showMessage() {
        // This is a comment
        let message = "Hello Student";
        console.log(message);
    }
</script>
```

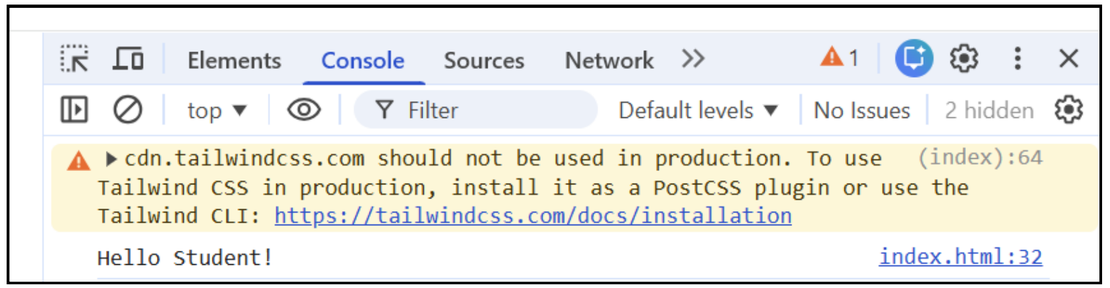
JavaScript Input & Output

JavaScript provides different ways to **display output** and **receive input** from users.

1. **Output Methods**
   - **console.log()**
     - Writes messages to the **browser console**
     - Mainly used for testing and debugging
     - **Example:**

```
console.log("Hello Student!");
```

*Output appears in **Developer Tools (F12 → Console)***



JavaScript Input & Output

JavaScript provides different ways to **display output** and **receive input** from users.

1. **Output Methods**
   - **document.write()**
     - Writes content directly into the **HTML page**
     - Mostly used for **simple demos (not recommended in real projects)**
     - **Example:**

```
document.write("<h2>Welcome to JavaScript</h2>");
```

Using it after page load will overwrite the page



Welcome to JavaScript

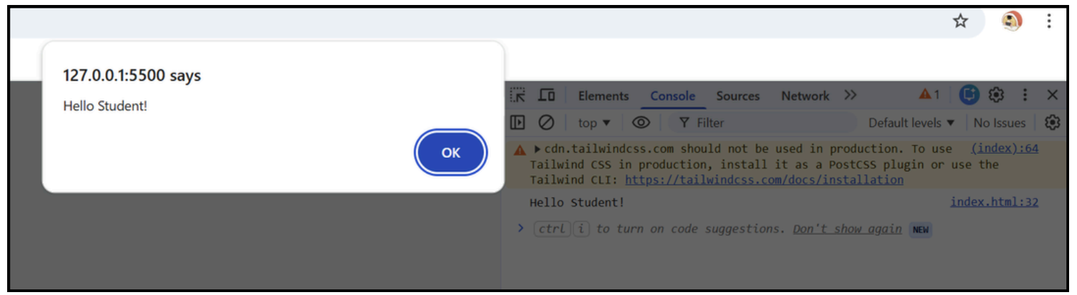
JavaScript Input & Output

JavaScript provides different ways to **display output** and **receive input** from users.

1. **Output Methods**
   - **alert()**
     - Displays a popup message box with an OK button
     - **Example:**

```
alert("Hello Student!");
```

Used to show important messages



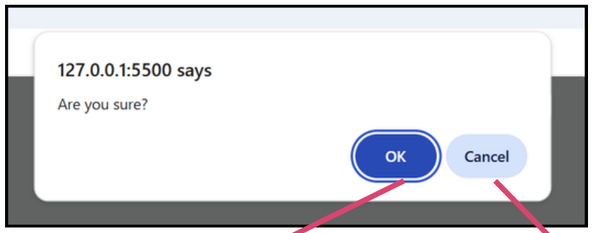
JavaScript Input & Output

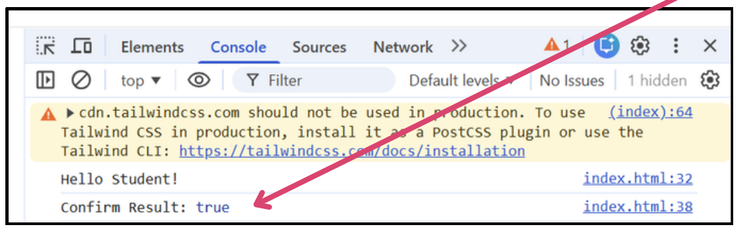
JavaScript provides different ways to **display output** and **receive input** from users.

2. **Input Methods**
   - **confirm()**
     - Shows a dialog box with **OK and Cancel**
     - Returns **true (OK)** or **false (Cancel)**
     - **Example:**

```
let confirmResult = confirm("Are you sure?");
console.log("Confirm Result:", confirmResult);
```

*Output appears in **Developer Tools (F12 → Console)***







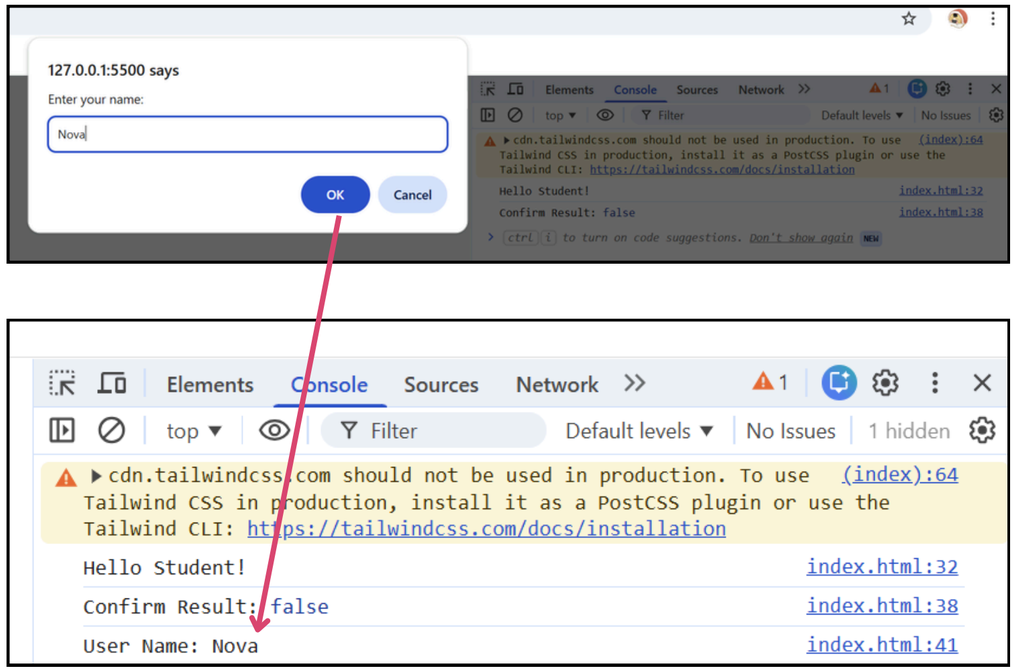
JavaScript Input & Output

JavaScript provides different ways to **display output** and **receive input** from users.

2. **Input Methods**
   - **prompt()**
     - Asks the user to **enter input**
     - Returns the entered value as a **string**
     - **Example:**

```
let userName = prompt("Enter your name:");
console.log("User Name:", userName);
```

*Output appears in **Developer Tools (F12 → Console)***



Variables in JavaScript

What is a Variable?

A **variable** is used to store data.

Declare Variables:

```
var name = "John";
let age = 20;
const PI = 3.14;
```

Important Rules:

- End statements with ;
- Use // or /* */ for comments

```
// This is a comment
let x = 10; /* inline comment */
```

Variables in JavaScript

var vs **let** vs **const**

```
var name = "John";
let age = 20;
const PI = 3.14;
```

var (Old way)

- Function scoped
- Can be redeclared
  - **Example**:

```
var x = 10;
var x = 20; // allowed
```

let (ES6)

- Block scoped **{ }**
- Cannot redeclare
  - **Example**:

```
let x = 10;
// let x = 20 , error
```

const (ES6)

- Block scoped
- Cannot change value
  - **Example**:

```
const PI = 3.14;
// PI = 3.15 , error
```

Variables in JavaScript

var vs let vs const

```
var name = "John";
let age = 20;
const PI = 3.14;
```

Keyword	Scope	Can Change	Use Case
var	function	Yes	old code
let	block	No	normal variable
const	block	No	fixed value

Data Types in JavaScript

In JavaScript there are 5 different data types that can contain values:

- string
- number
- boolean
- object
- function

2 data types that cannot contain values:

- null
- undefined

There are 6 types of objects:

- Object
- Date
- Array
- String
- Number
- Boolean

Special Types

- object
- function

```
typeof null "object" // (JavaScript bug)
```

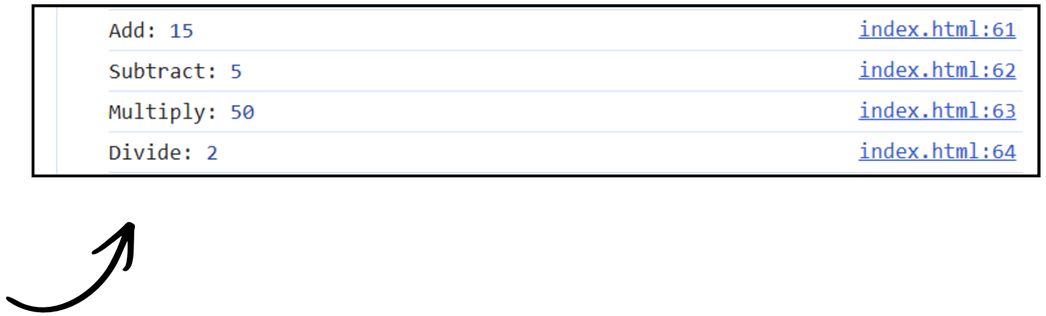
Operators in JavaScript

Operators are used to **perform operations on values.**

1. **Arithmetic Operators**

```
let a = 10;
let b = 5;

console.log("Add:", a + b);
console.log("Subtract:", a - b);
console.log("Multiply:", a * b);
console.log("Divide:", a / b);
```



Operators in JavaScript

Operators are used to **perform operations on values.**

2. **Assignment Operators**

```
let x = 10;
x += 5;  // x = x + 5
console.log("x =", x);
```



```
x = 15                                                    index.html:69
```

Operators in JavaScript

Operators are used to **perform operations on values.**

3. **Comparison Operators**

```
console.log(10 > 5);        // true
console.log(10 == "10");    // true
console.log(10 === "10");   // false (strict)
```

Note:

- *=== checks VALUE + TYPE*
  - *10 → Number*
  - *"10" → String*
- *== only checks value, not type*
  - *JavaScript automatically converts "10" → 10*



```
true                                    index.html:72
true                                    index.html:73
false                                   index.html:74
```

Operators in JavaScript

Operators are used to **perform operations on values.**

4. **Logical Operators**

```
console.log(true && false);
console.log(true || false);
console.log(!true);
```



```
false                                    index.html:77
true                                     index.html:78
false                                    index.html:79
```

Operators in JavaScript

Type Operators :

typeof — Check data type

```
let name = "John";
console.log(typeof name); // string
```

instanceof — Check object type

```
let arr = [1, 2, 3];
console.log(arr instanceof Array); // true
```

Operators in JavaScript

Comparison Operators

Operator	Name	Description	Example	Result
==	Equal to	Checks if values are equal (may ignore type in some languages like JS)	5 == "5"	TRUE
===	Strict equal to	Checks if values AND types are equal	5 === "5"	FALSE
!=	Not equal	Checks if values are not equal	5 != 3	TRUE
>	Greater than	Checks if left value is greater than right	10 > 5	TRUE
<	Less than	Checks if left value is less than right	3 < 8	TRUE
>=	Greater than or equal	Checks if left value is greater than or equal	5 >= 5	TRUE
<=	Less than or equal	Checks if left value is less than or equal	4 <= 6	TRUE

Operators in JavaScript

Logical Operators

Operator	Name	Description	Example	Result
&&	Logical AND	True if both conditions are true	(5 > 3 && 10 > 8)	true
||	Logical OR	True if at least one condition is true	(5 > 10 || 8 > 3)	true
!	Logical NOT	Reverses the result	!(5 > 3)	false

Spread Operator (ES6)

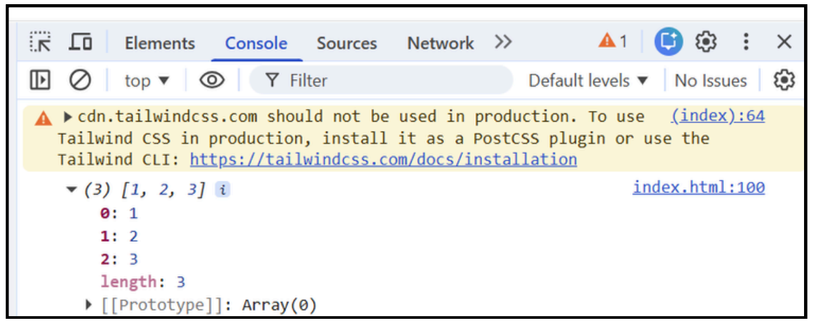
What is Spread?

Used to **copy or expand** arrays/objects

Array Example:

```
let arr1 = [1, 2, 3];
let arr2 = [...arr1];

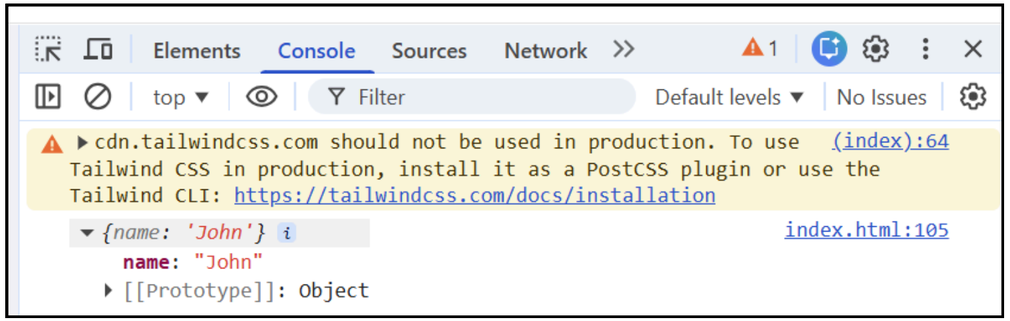
console.log(arr2);
```



Opject Example:

```
let obj1 = { name: "John" };
let obj2 = { ...obj1 };

console.log(obj2);
```



Control Statements in JavaScript

Control statements are used to control the flow of execution in a program.

They decide:

- Which code runs
- When it runs
- How many times it runs

Control Statements in JavaScript

Types of Control Statements

JavaScript has **2 main types of control statements:**

1. **Conditional Statements (Decision Making)** — Used to make decisions based on conditions
   - if
   - if...else
   - if...else if...else
   - switch
2. **Loop Statements (Repetition)** — Used to repeat code multiple times
   - for
   - while
   - do...while
   - for...in
   - for...of

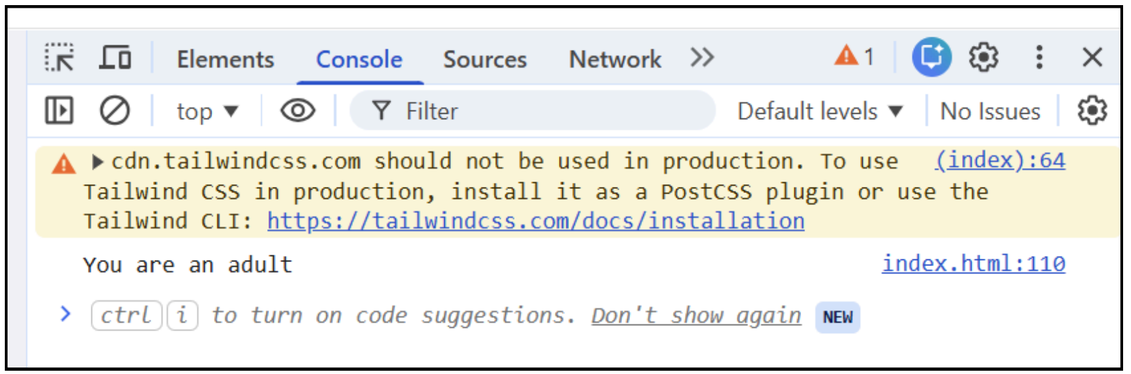
Control Statement and Structure

Conditional Statements (Decision Making)

1. if Statement — Executes code only if condition is true

```
let age = 18;

if (age >= 18) {
    console.log("You are an adult");
}
```



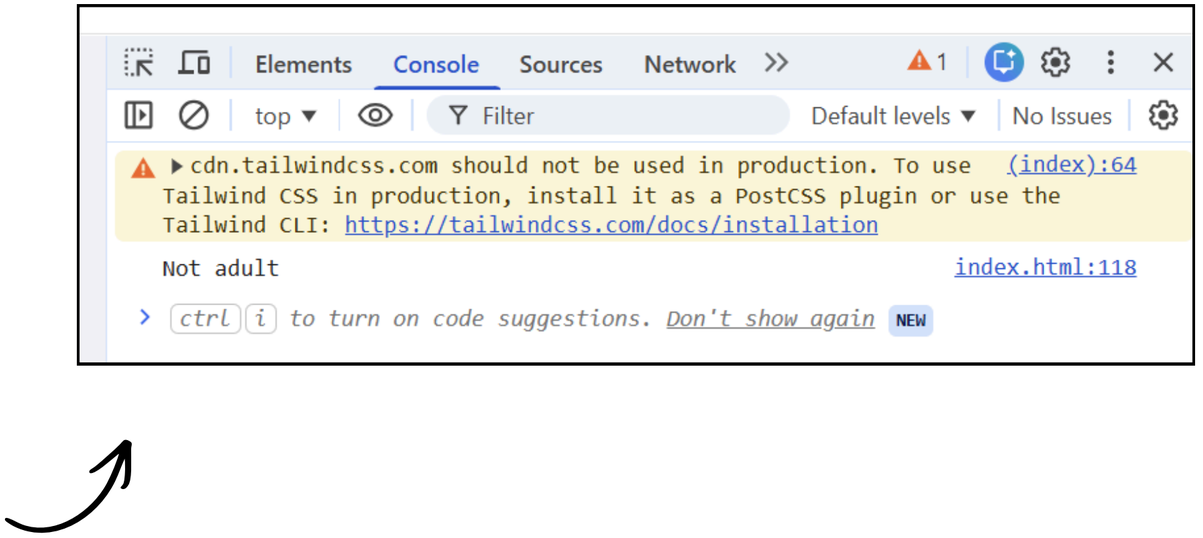
Control Statement and Structure

Conditional Statements (Decision Making)

2. if...else — Choose between two options

```
let age = 16;

if (age >= 18) {
    console.log("Adult");
} else {
    console.log("Not adult");
}
```



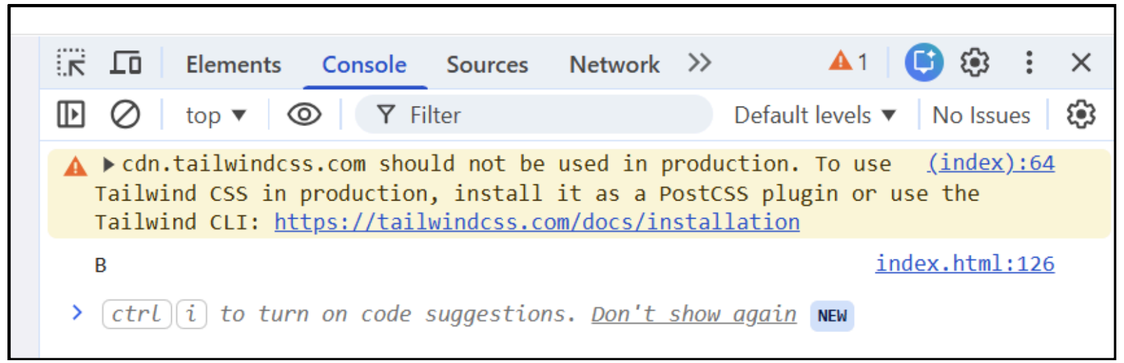
Control Statement and Structure

Conditional Statements (Decision Making)

3. if...else if...else — Multiple conditions

```
let score = 85;

if (score >= 90) {
    console.log("A");
} else if (score >= 80) {
    console.log("B");
} else {
    console.log("C");
}
```



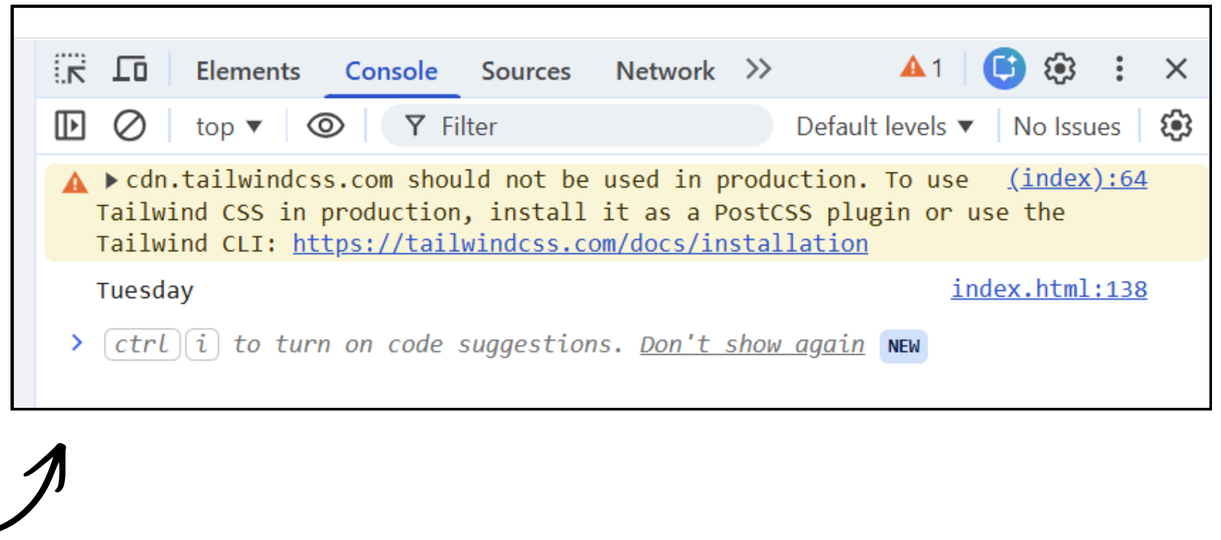
Control Statement and Structure

Conditional Statements (Decision Making)

4. switch Statement — Used when checking **multiple fixed values**

```
let day = 2;

switch (day) {
    case 1:
        console.log("Monday");
        break;
    case 2:
        console.log("Tuesday");
        break;
    default:
        console.log("Unknown");
}
```



Control Statement and Structure

if **vs** switch

if...else	switch
Works with ranges (>, <, >=)	Works with exact values
More flexible	Cleaner for many cases
Can handle complex logic	Best for simple comparisons

Example :

```
// if (range)
if (score > 80)
```

```
// switch (exact)
switch (day)
```

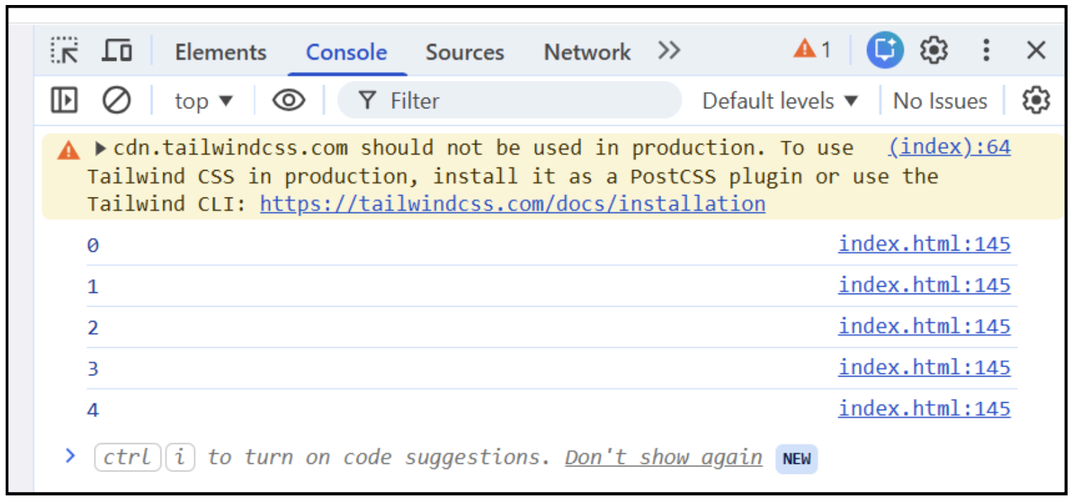
Control Statement and Structure

Loops (Control Structure)

1. for Loop — Used when you know how many times to run

```
for (let i = 0; i < 5; i++) {
    console.log(i);
}
```





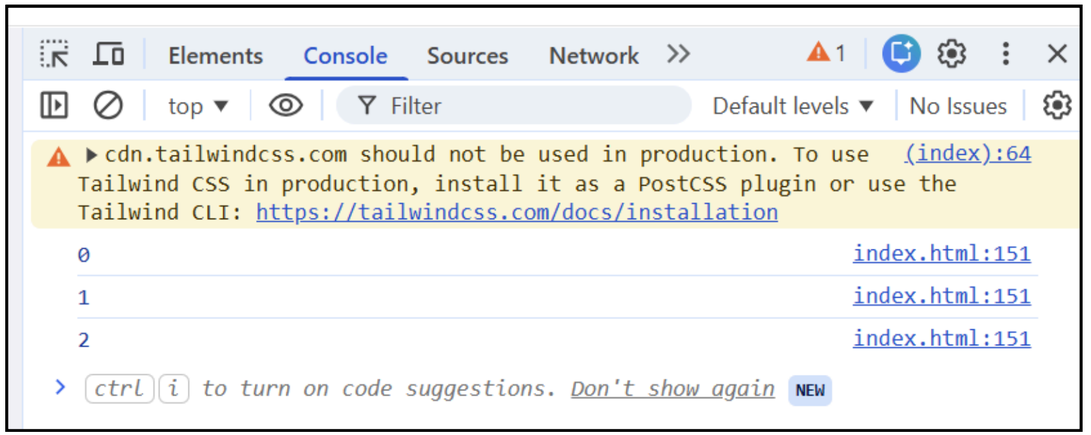
Control Statement and Structure

Loops (Control Structure)

1. while Loop — Runs while condition is true

```
let i = 0;

while (i < 3) {
    console.log(i);
    i++;
}
```



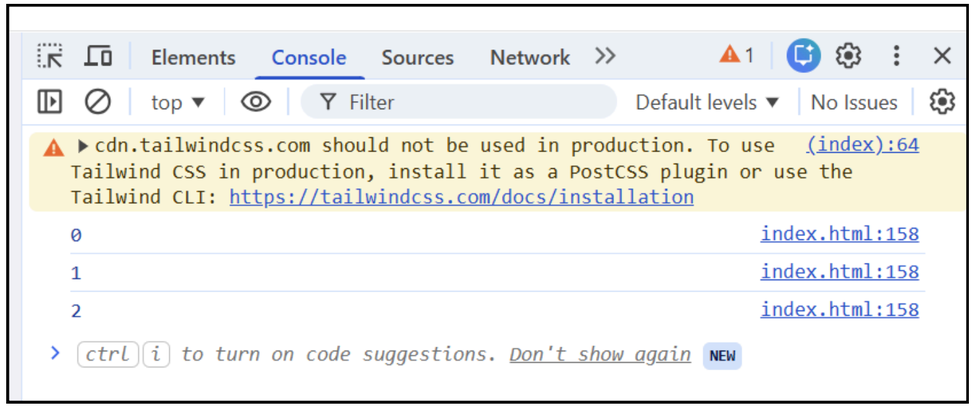
Control Statement and Structure

Loops (Control Structure)

1. do...while Loop — Runs at least once

```
let i = 0;

do {
    console.log(i);
    i++;
} while (i < 3);
```



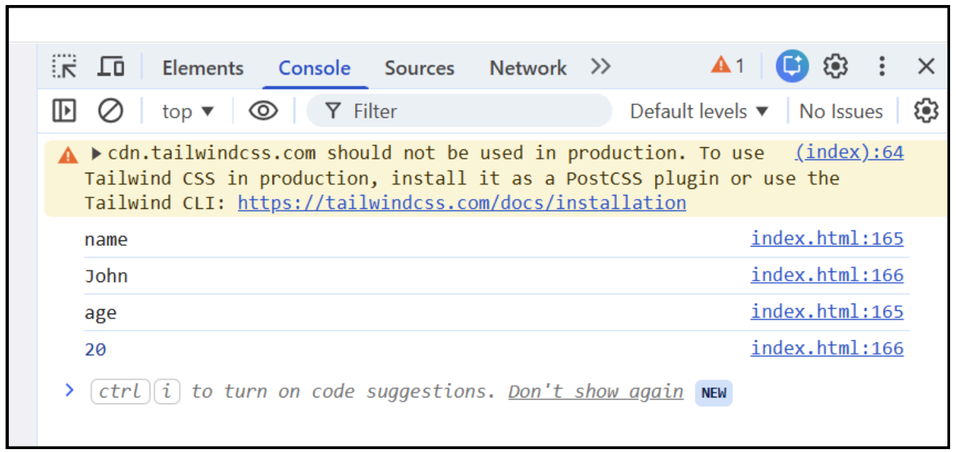
Control Statement and Structure

Loops (Control Structure)

1. for...in Loop — Used for **objects (keys/indexes)**

```
let user = { name: "John", age: 20 };

for (let key in user) {
    console.log(key);      // name, age
    console.log(user[key]); // John, 20
}
```



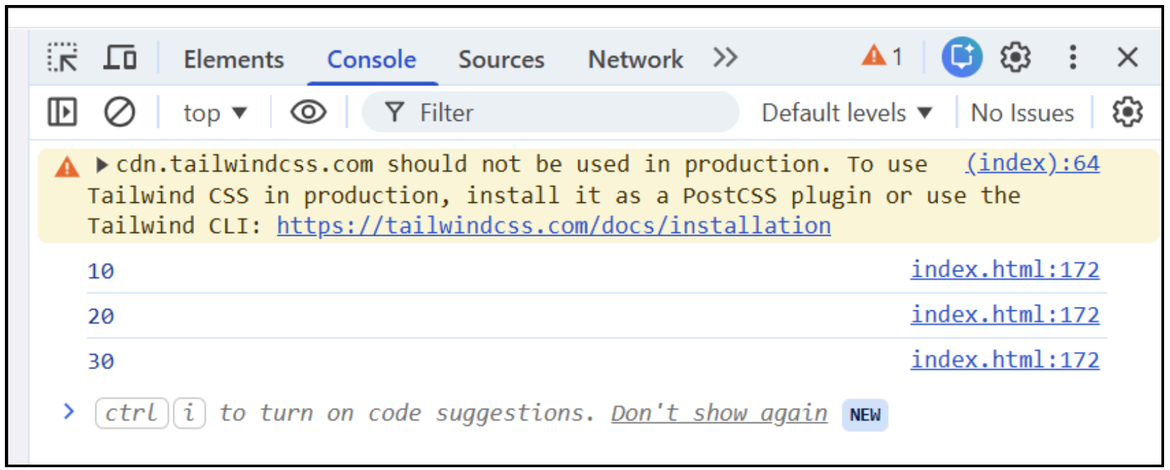
Control Statement and Structure

Loops (Control Structure)

1. for...of Loop — Used for **arrays (values)**

```
let arr = [10, 20, 30];

for (let value of arr) {
    console.log(value);
}
```



Control Statement and Structure

Use if → for logic and conditions
Use switch → for menu / fixed values
Use for → when count is known
Use while → when condition-based
Use for...in → objects
Use for...of → arrays

Function in JavaScript

What is a Function?

A function is a **block of code designed to perform a specific task.**

It runs only when called

Syntax :

```
function sayHello() {
    console.log("Hello!");
}
```

Call function:

```
sayHello()
```

Function in JavaScript

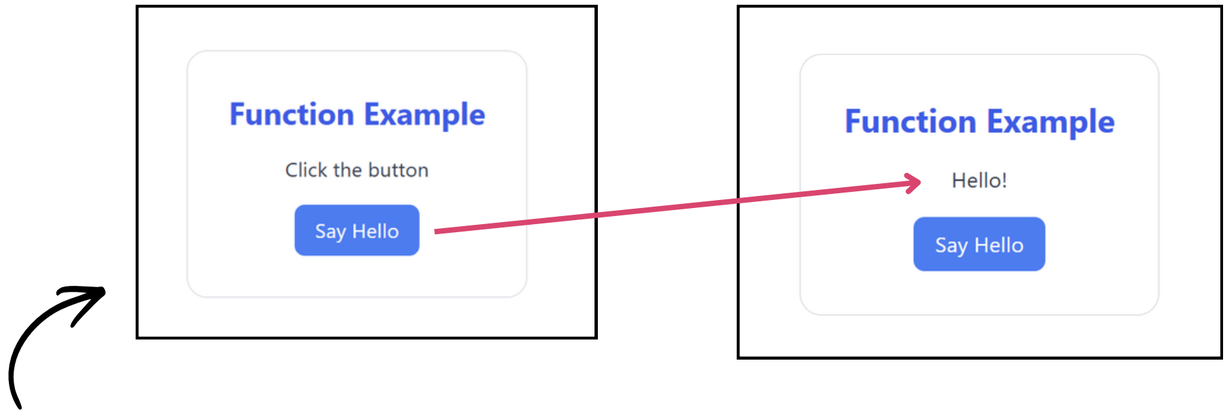
What is a Function?

A function is a **block of code designed to perform a specific task.**

It runs only when called

Example :

```
<script>
    function sayHello() {
        console.log("Hello!");
        document.getElementById("output").innerText = "Hello!";
    }
</script>
```




```
<button onclick="sayHello()" class="bg-blue-500 hover:bg-blue-600 text-white px-4 py-2 rounded-lg transition">
    Say Hello
</button>
```

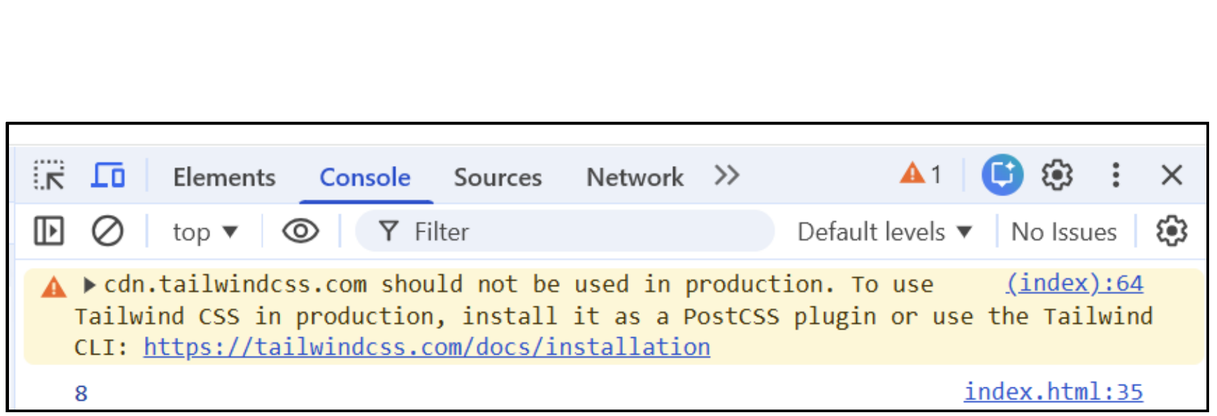
Function in JavaScript

Function with Parameters & Return

Example:

```
function add(a, b) {
    return a + b;
}

let result = add(5, 3);
console.log(result);
```



Function in JavaScript

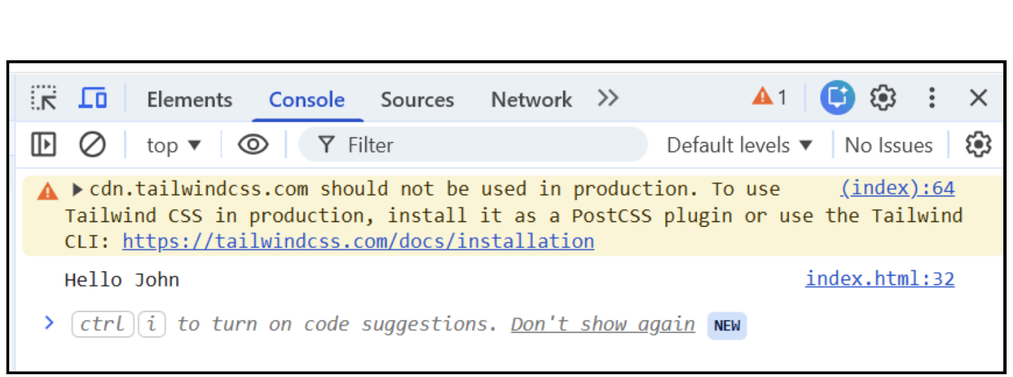
Callback Function — passed into another function and executed later

Example:

```
function greet(name) {
    console.log("Hello " + name);
}

function processUser(callback) {
    let name = "John";
    callback(name);
}

processUser(greet);
```



Function inside another function

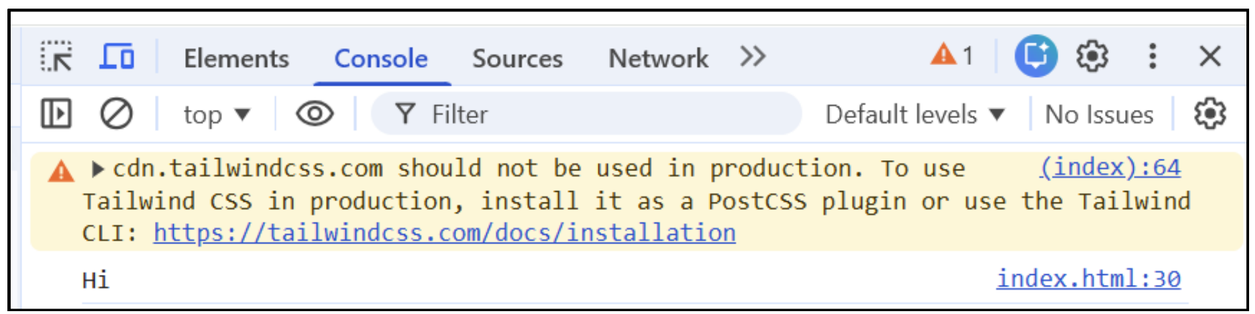
Function in JavaScript

Anonymous Function — Function **without a name**

Example:

```
let sayHi = function () {
    console.log("Hi");
};

sayHi();
```



Function in JavaScript

Arrow Function (ES6) — Shorter way to write functions

Syntax:

```
// Arrow function
let add = (a, b) => {
    return a + b;
};
```

- **add** = function name
- **(a, b)** = parameters
- **return a + b;** = sends back the result

Function in JavaScript

Arrow Function (ES6) — Shorter way to write functions

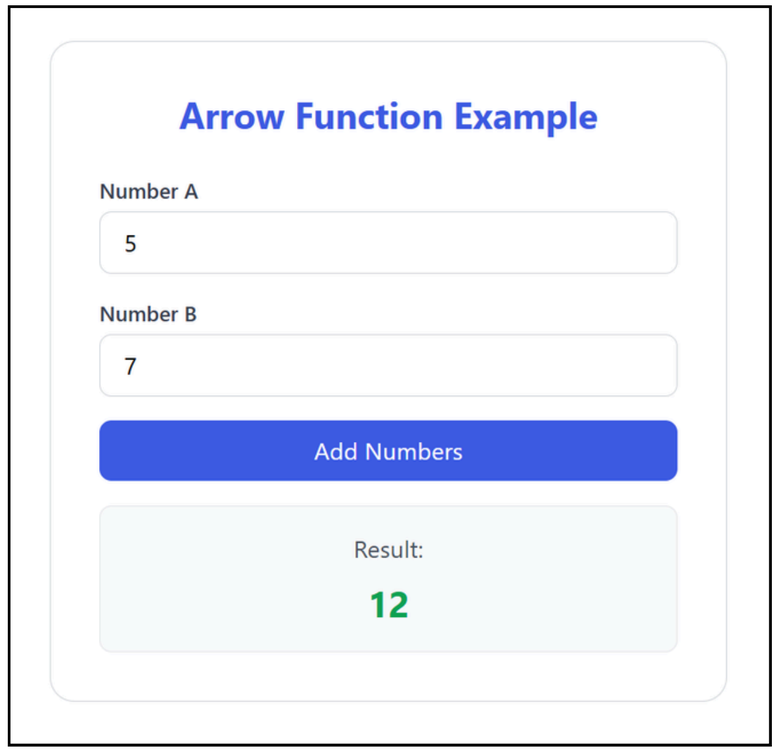
Example:

```
// Arrow function
let add = (a, b) => {
    return a + b;
};

function calculateSum() {
    let a = Number(document.getElementById("num1").value);
    let b = Number(document.getElementById("num2").value);

    let total = add(a, b);

    document.getElementById("result").innerText = total;
}
```



Function in JavaScript

Short Arrow Function

Single Line

```
let add = (a, b) => a + b;
```

One Parameter

```
let square = x => x * x;
```

Function in JavaScript

Built-in Functions

JavaScript provides **built-in objects:**

- String
- Number
- Math

Function in JavaScript

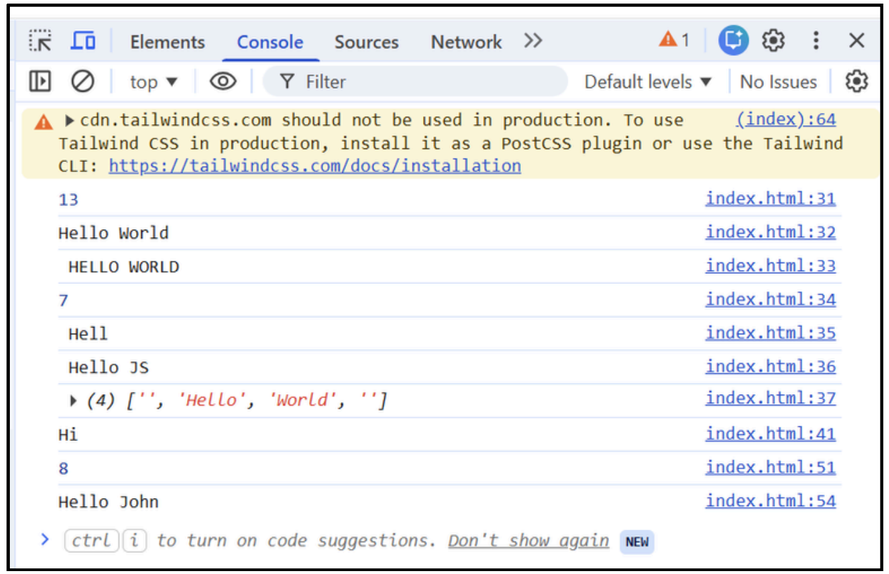
Built-in Functions

JavaScript provides **built-in objects:**

- **String Functions**

```
let text = " Hello World ";

console.log(text.length);         // length
console.log(text.trim());         // remove spaces
console.log(text.toUpperCase()); // uppercase
console.log(text.indexOf("World"));
console.log(text.substring(0, 5));
console.log(text.replace("World", "JS"));
console.log(text.split(" "));
```



Function in JavaScript

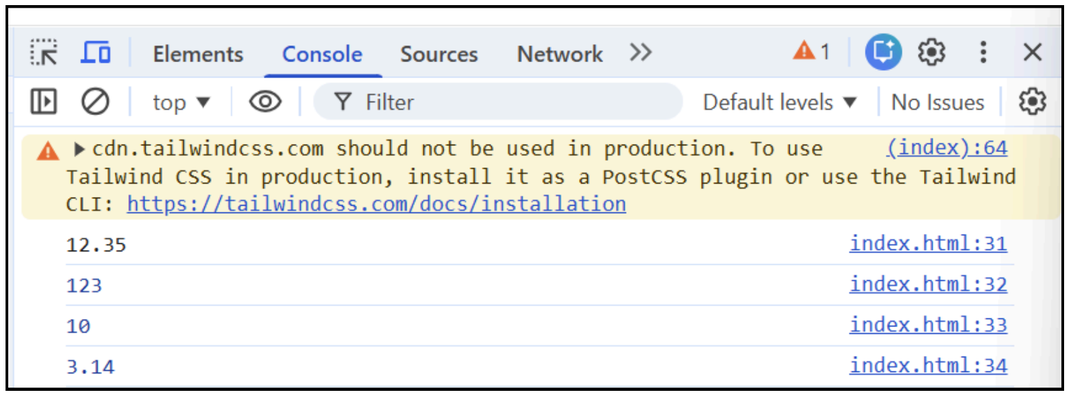
Built-in Functions

JavaScript provides **built-in objects:**

- **Number Functions**

```
let num = 12.345;

console.log(num.toFixed(2));   // 12.35
console.log(Number("123"));    // 123
console.log(parseInt("10px")); // 10
console.log(parseFloat("3.14"));
```



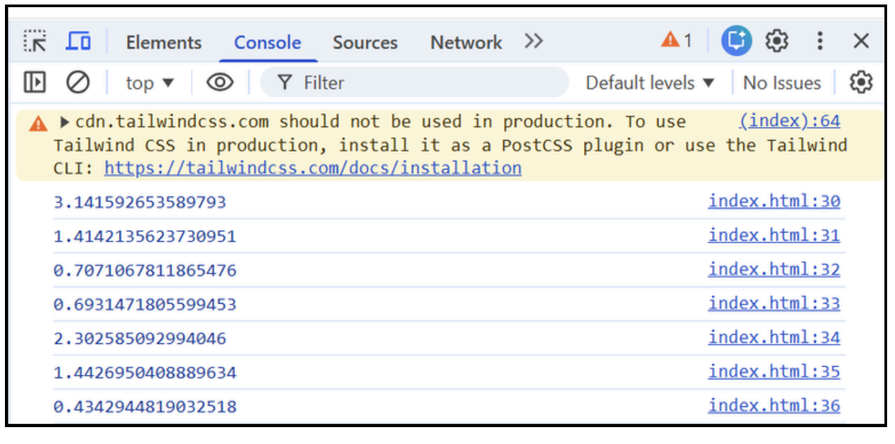
Function in JavaScript

Built-in Functions

JavaScript provides **built-in objects:**

- **Math Object**
  - Properties

```
console.log(Math.PI);       // returns PI
console.log(Math.SQRT2);    // returns the square root of 2
console.log(Math.SQRT1_2);  // returns the square root of 1/2
console.log(Math.LN2);      // returns the natural logarithm of 2
console.log(Math.LN10);     // returns the natural logarithm of 10
console.log(Math.LOG2E);    // returns the base-2 logarithm of E
console.log(Math.LOG10E);   // returns the base-10 logarithm of E
```



Function in JavaScript

Built-in Functions

JavaScript provides **built-in objects:**

- **Math Object**
  - Methods

Method	Description
Math.round(x)	Returns x rounded to its nearest integer
Math.ceil(x)	Returns x rounded up to its nearest integer
Math.floor(x)	Returns x rounded down to its nearest integer
Math.trunc(x)	Returns the integer part of x (new in ES6)

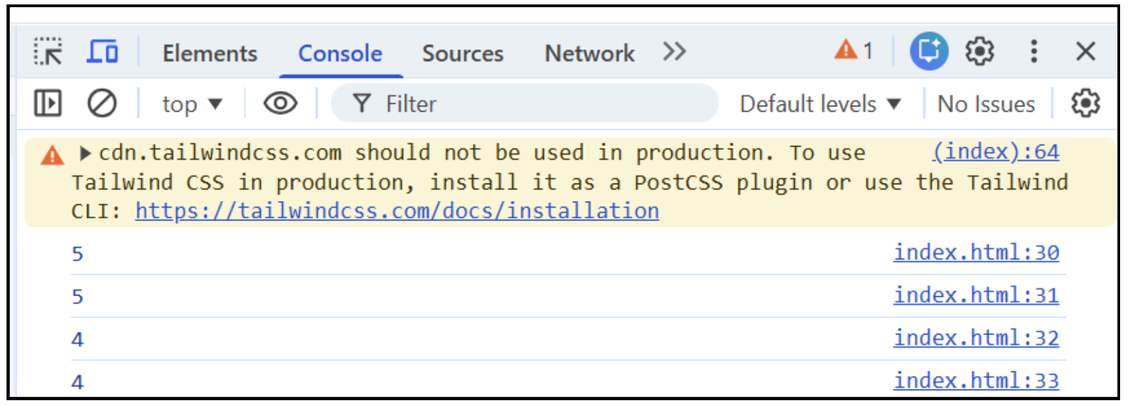
Function in JavaScript

Built-in Functions

JavaScript provides **built-in objects:**

- **Math Object**
  - Methods
    - Example:

```
console.log(Math.round(4.6)); // 5
console.log(Math.ceil(4.2));  // 5
console.log(Math.floor(4.8)); // 4
console.log(Math.trunc(4.9)); // 4
```



Array in JavaScript

What is an Array?

An **array** is a special variable that can store **multiple values in one variable**

Instead of this:

```
let a = 10;
let b = 20;
let c = 30;
```



You can use:

```
let numbers = [10, 20, 30];
```

Array in JavaScript

What is an Array?

An **array** is a special variable that can store **multiple values in one variable**

Array Syntax:

```
let arr = [item1, item2, item3];
```

Example:

```
let fruits = ["Apple", "Banana", "Mango"];
```



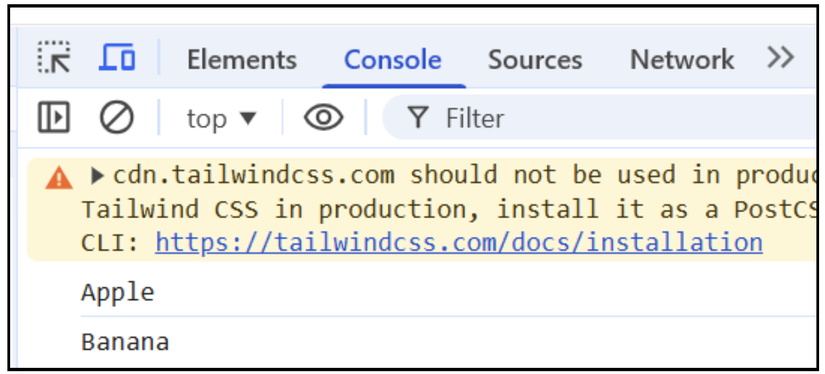
Array in JavaScript

What is an Array?

An **array** is a special variable that can store **multiple values in one variable**

Accessing Array Elements : Use **index (position)**

```
let fruits = ["Apple", "Banana", "Mango"];
console.log(fruits[0]); // Apple
console.log(fruits[1]); // Banana
```



Note:

- Index starts at 0
- First item = index 0

Array in JavaScript

What is an Array?

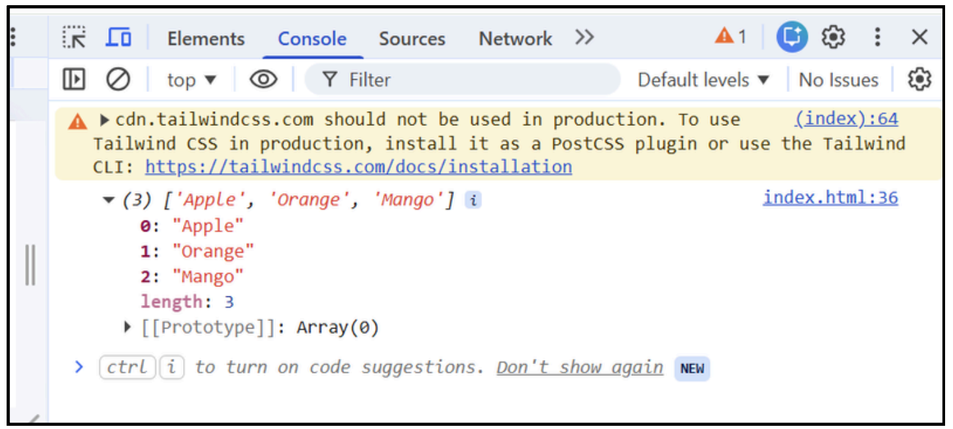
An **array** is a special variable that can store **multiple values in one variable**

Modify Array

```
let fruits = ["Apple", "Banana", "Mango"];

    fruits[1] = "Orange";

    console.log(fruits);
```



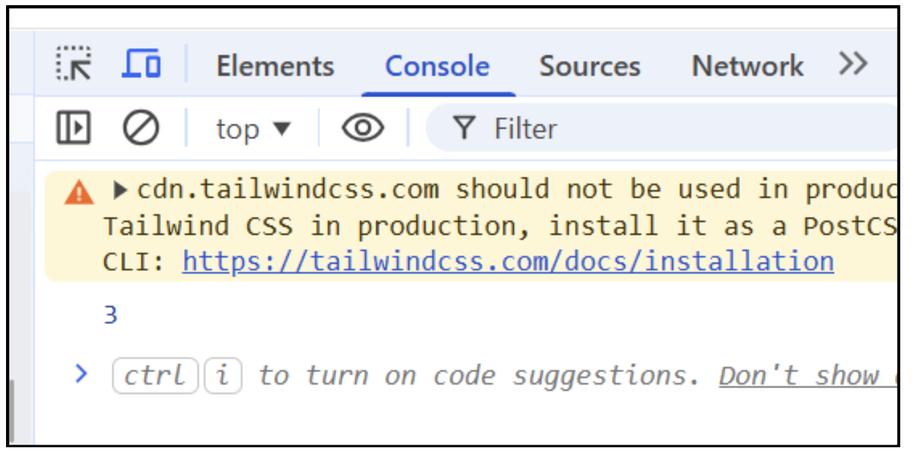
Array in JavaScript

What is an Array?

An **array** is a special variable that can store **multiple values in one variable**

Array Length

```
let fruits = ["Apple", "Banana", "Mango"];
console.log(fruits.length);
```



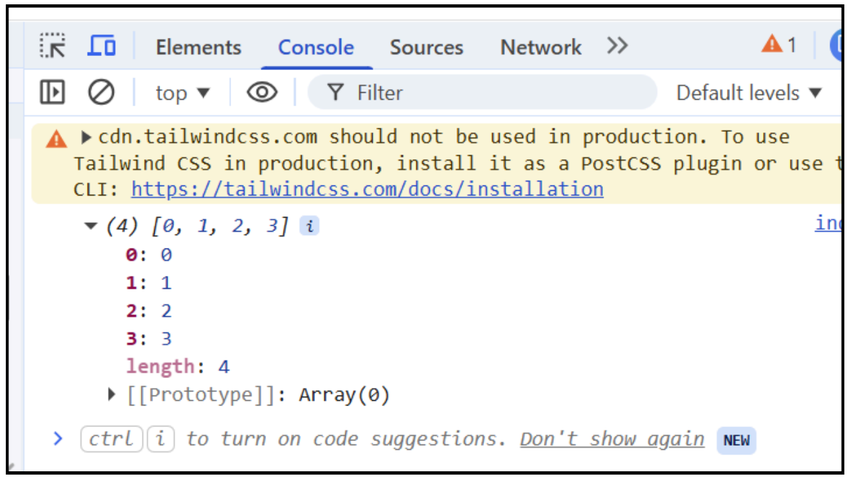
Array in JavaScript

Array Methods

- **Add Items**

```
let arr = [1, 2];

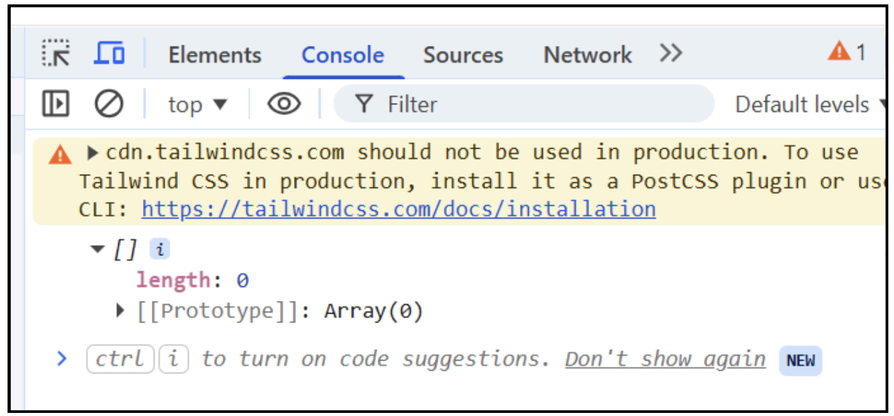
arr.push(3);   // add at end
arr.unshift(0); // add at beginning
```



- **Remove Items**

```
let arr = [1, 2];

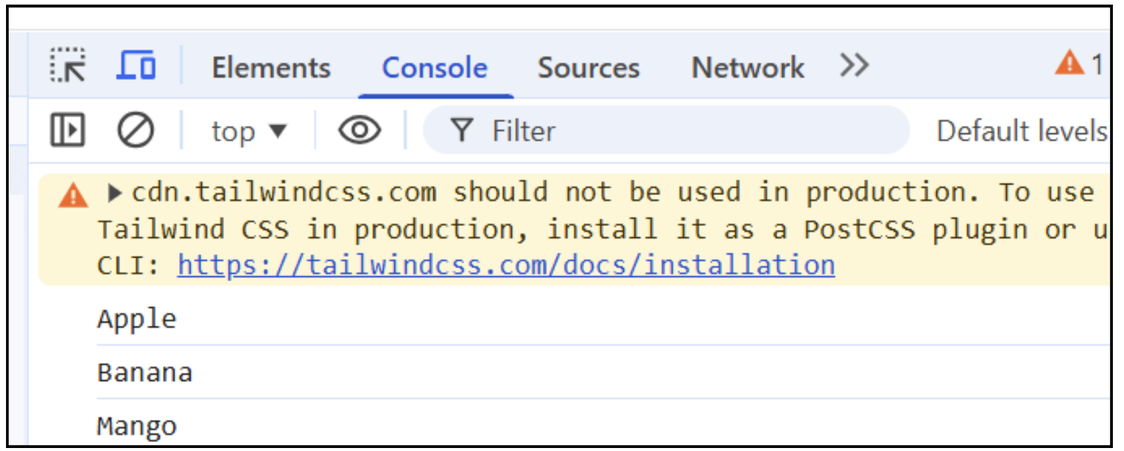
arr.pop();     // remove last
arr.shift();   // remove first
```



Array in JavaScript

Loop Through Array

```
let fruits = ["Apple", "Banana", "Mango"];
for (let fruit of fruits) {
    console.log(fruit);
}
```

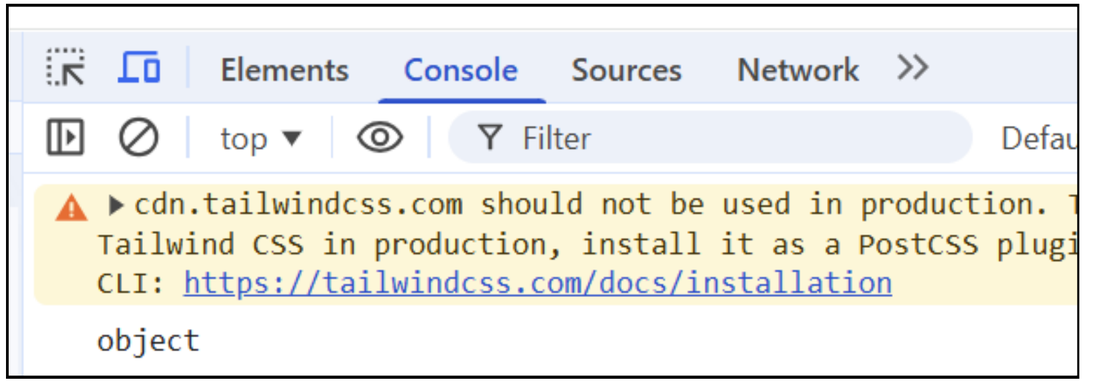


Array is an Object

In JavaScript:

```
let arr = [1, 2, 3];

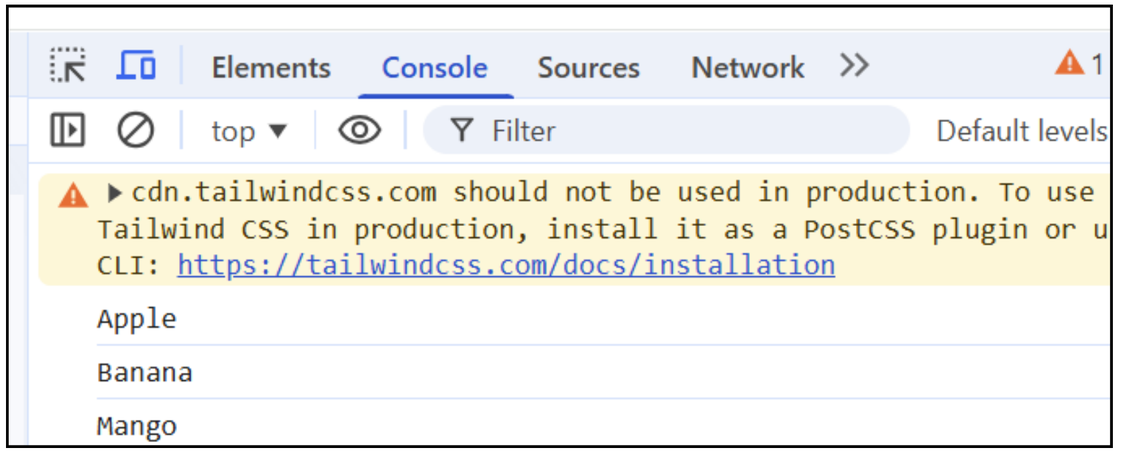
console.log(typeof arr); // object
```



Array in JavaScript

Loop Through Array

```
let fruits = ["Apple", "Banana", "Mango"];
for (let fruit of fruits) {
    console.log(fruit);
}
```



Array is an Object

In JavaScript:

```
var ARR_NAME = { KEY: VALUE };
```

```
// Array
let arr = [10, 20, 30];

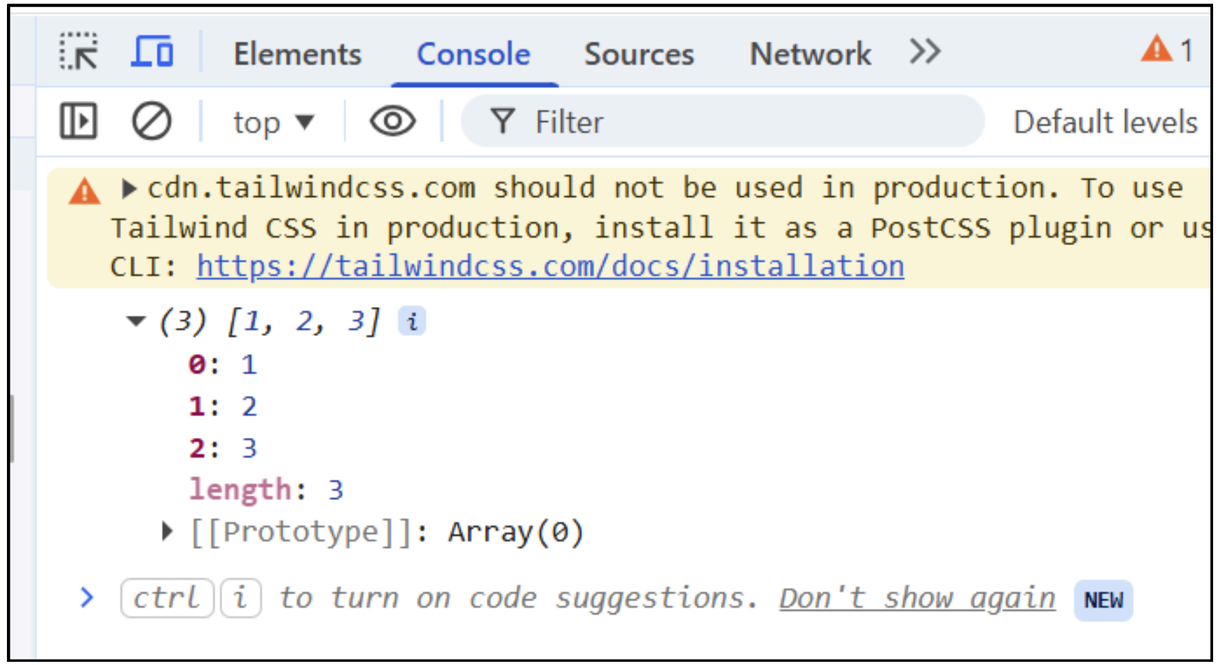
// Object
let obj = { name: "John", age: 20 };
```

Array in JavaScript

Spread Operator with Array

```
let arr1 = [1, 2, 3];
let arr2 = [...arr1];

console.log(arr2);
```







THANK YOU

For your attention !

